Lab 6 — Code Coverage Report

Software Engineering Lab | shipping_quote.py | SETAPESU26/L6_L7_04_shipping_quotation

Objective

Measure and improve the line (statement) coverage of shipping_quote.py, which implements process_shipments() and monitor_container_pressures(), using Python's coverage library and pytest.

Tools and Commands

- coverage.py 7.16.0, pytest 9.1.1
- Run: python -m coverage run --branch -m pytest test_shipping_quote.py
- Report: python -m coverage report -m

Baseline Coverage

With only the single starter test (test_single_ground_shipment), shipping_quote.py had substantial untested code, including the entire monitor_container_pressures() function and most of the rejection, hazmat, and status-selection logic in process_shipments().

Module	Statements	Missed	Coverage
shipping_quote.py	53	27	45%

Missing lines included input validation (3), manifest rejection (10), transport-mode branching (17, 21-22), zero-weight rejection (31-32), hazmat handling (35-39, 41), express pricing (46), status selection (51, 53), and the whole monitor_container_pressures function (58-70).

Test Cases Added

Fourteen additional test cases were written to exercise every previously-unreached statement:

- Invalid input: non-list shipments and negative handling_fee raise ValueError
- A shipment with manifest_approved = False is skipped
- Air-mode shipment pricing (base_rate = 12.0)
- An unrecognised transport mode is rejected
- A package with weight_kg = 0 is rejected
- A hazmat package on a fragile shipment zeroes the shipment cost
- A hazmat package on a non-fragile shipment adds the flat surcharge
- A package over 30kg is charged the 1.5x heavy-weight rate
- express_priority applies the 1.25x multiplier
- More than three rejected parcels returns FLAGGED_REVIEW
- An empty pressure_logs list returns NO_DATA
- Two consecutive low-pressure readings return LEAK_DETECTED
- A stable set of readings returns STABLE

Final Result

After adding the fourteen test cases (fifteen total), every statement in shipping_quote.py executes at least once.

Module	Statements	Missed	Coverage
shipping_quote.py	53	0	100%
test_shipping_quote.py	59	0	100%
TOTAL	112	0	100%

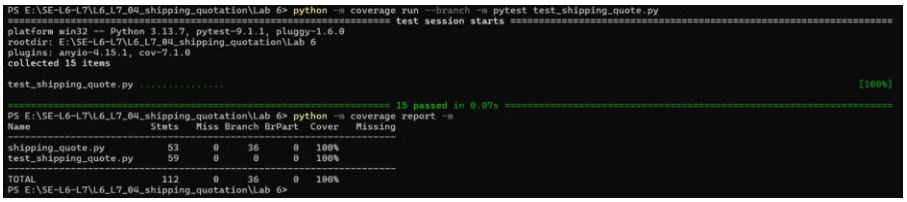


Figure 1. Terminal output — 15 tests passing, 100% statement coverage.

Conclusion

Line coverage improved from 45% to 100%, exceeding the 90% target. Every statement in shipping_quote.py, including both business functions, is now exercised by at least one test.